

PROSTACKHUB

Software Company | IT Consultancy | Online Services

CYBERSECURITY INTERNSHIP

Task Completion Report

Task: Password Strength & Breach Checker

Intern Name:	Muhammad Maaz Afridi
Internship Track:	Cybersecurity Internship
Organization:	ProStackHub
Task Assigned:	Task - Password Strength & Breach Checker
GitHub Repository:	ProStackHub_PasswordChecker
GitHub Link:	github.com/maazafridii182-sketch/ProStackHub_PasswordChecker
Report Date:	July 28, 2026

ProStackHub | Software Company | IT Consultancy | Online Services
Reg. Office: Chhapra, Bihar - 841412, INDIA | UDYAM: UDYAM-BR-31-0056727
hello@prostackhub.com | +91 7850057539 | www.prostackhub.com

1. Project Overview

This report documents the completion of **Task: Password Strength & Breach Checker**, one of the four tasks offered under the ProStackHub Cybersecurity Internship track. The objective of this task was to build an application that evaluates how strong a user's password is, checks whether that password has previously appeared in any known data breach, and provides a way to generate new, secure passwords.

The final application was developed in Python, tested on a Kali Linux virtual machine, and published as a public repository on GitHub, in line with the internship submission guidelines.

2. Objective of the Task

- Evaluate password strength using entropy-based analysis (via the zxcvbn library).
- Check whether a password has been exposed in known data breaches, without ever sending the plain-text password over the internet.
- Implement the Have I Been Pwned (HIBP) API using the k-anonymity model for privacy-safe breach lookups.
- Provide a secure, configurable random password generator.
- Deliver the project as a working command-line application, along with source code and documentation.

3. Tools & Technologies Used

Category	Tools & Technologies
Programming Language	Python 3.13
Strength Scoring	zxcvbn
Breach Lookup	Have I Been Pwned (HIBP) API, hashlib (SHA-1), k-Anonymity model
Networking	requests library
Password Generation	Python secrets module
Operating Environment	Kali Linux (VMware Workstation)
Version Control	Git + GitHub (SSH authentication)

4. How the Application Works

4.1 Password Strength Check

The application uses the **zxcvbn** library, which analyzes a password based on its length, common patterns, dictionary words, and character variety. It returns a score from 0 (Very Weak) to 4 (Very Strong), an estimated time to crack the password, and suggestions for improvement.

4.2 Breach Check using HavelBeenPwned API (k-Anonymity)

To check whether a password has appeared in a known data breach, the application never sends the plain password to any server. Instead, it follows these steps:

- The password is hashed locally using SHA-1.
- Only the first 5 characters of the resulting hash (the "prefix") are sent to the HIBP API.
- The API responds with a list of all hash suffixes that share that same prefix.
- The application checks locally whether its own hash suffix appears in that list.

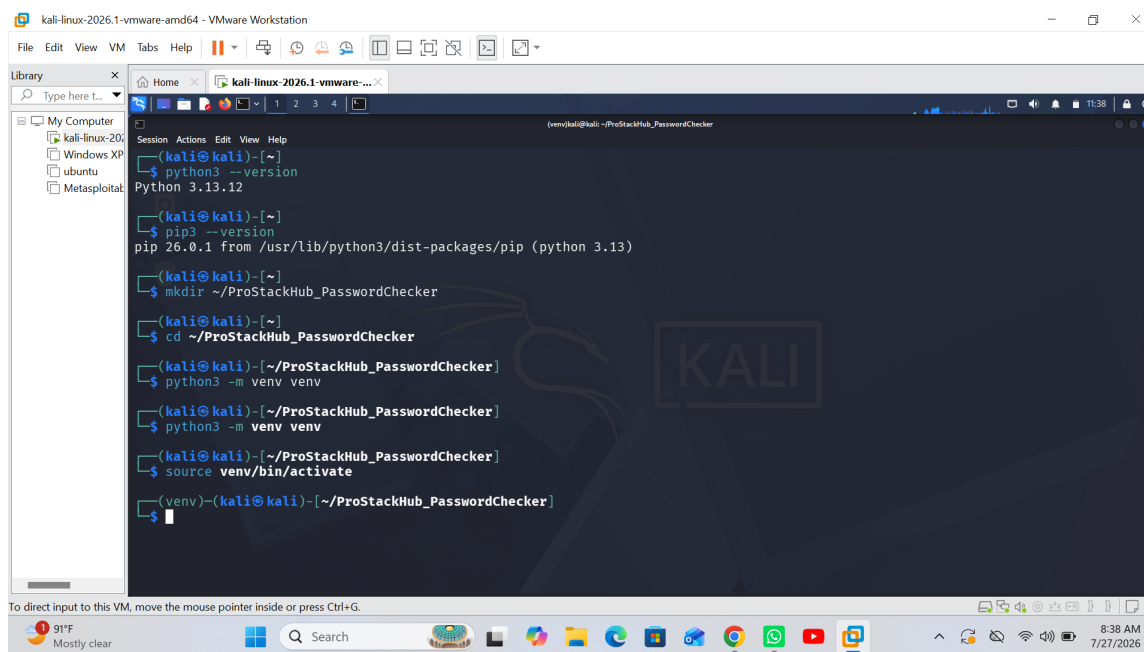
This technique, called **k-anonymity**, ensures that the real password (or its full hash) is never exposed to the network, while still allowing an accurate breach lookup.

4.3 Secure Password Generator

The application also includes a password generator built using Python's **secrets** module, which is designed for generating cryptographically strong random values. Users can specify a custom password length, or use the default of 16 characters.

5. Development Environment Setup

The project was developed inside a Kali Linux virtual machine (VMware Workstation). A dedicated Python virtual environment was created for the project to keep its dependencies isolated from the system-wide Python installation, and the required libraries (zxcvbn, requests) were installed inside it.

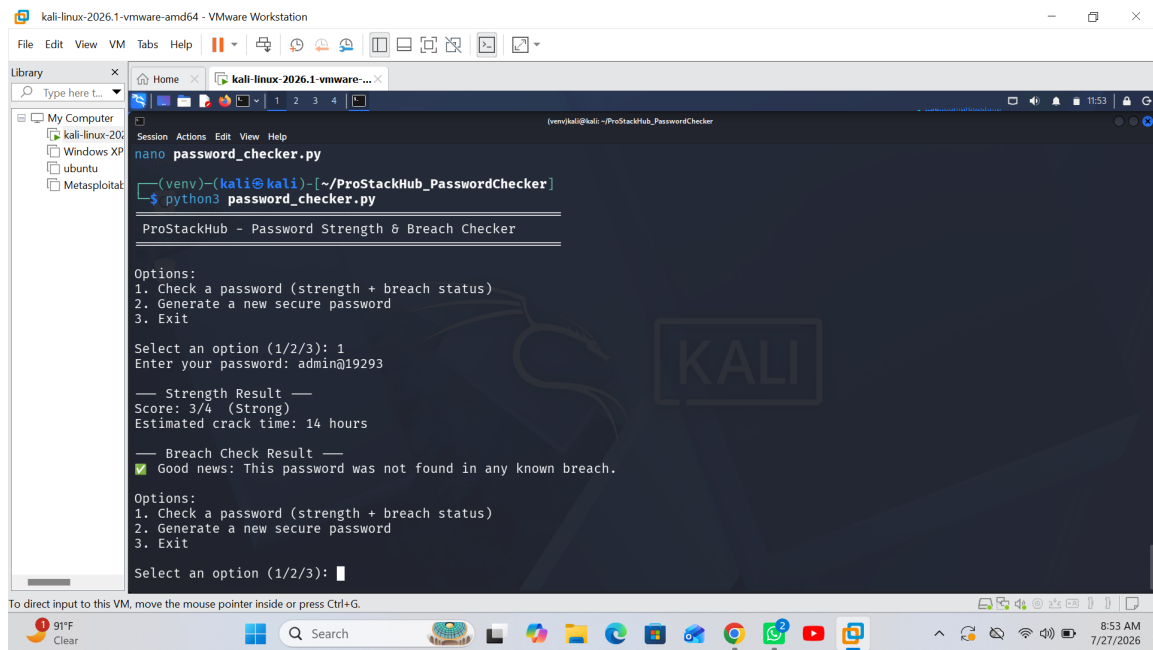


```
kali-linux-2026.1-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
Library
Type here to search
My Computer
kali-linux-2026.1-vmware-amd64
Windows XP
ubuntu
Metasploit
kali-linux-2026.1-vmware-amd64
kali@kali:~$ python3 --version
Python 3.13.12
kali@kali:~$ pip3 --version
pip 26.0.1 from /usr/lib/python3/dist-packages/pip (python 3.13)
kali@kali:~$ mkdir ~/ProStackHub_PasswordChecker
kali@kali:~$ cd ~/ProStackHub_PasswordChecker
kali@kali:~/ProStackHub_PasswordChecker$ python3 -m venv venv
kali@kali:~/ProStackHub_PasswordChecker$ python3 -m venv venv
kali@kali:~/ProStackHub_PasswordChecker$ source venv/bin/activate
(venv)kali@kali:~/ProStackHub_PasswordChecker$
```

Screenshot: Verifying Python and pip installation, then creating and activating the Python virtual environment (venv) inside the project folder.

6. Application Testing

The application was tested directly in the terminal to confirm that all three features worked correctly: password strength scoring, live breach lookup, and password generation.



```
kali-linux-2026.1-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
Library
Type here to search
My Computer
kali-linux-2026.1-vmware-amd64
Windows XP
ubuntu
Metasploitat
kali-linux-2026.1-vmware-amd64
Session Actions Edit View Help
nano password_checker.py
(venv)~(kali@kali)~[~/ProStackHub_PasswordChecker]
$ python3 password_checker.py
ProStackHub - Password Strength & Breach Checker

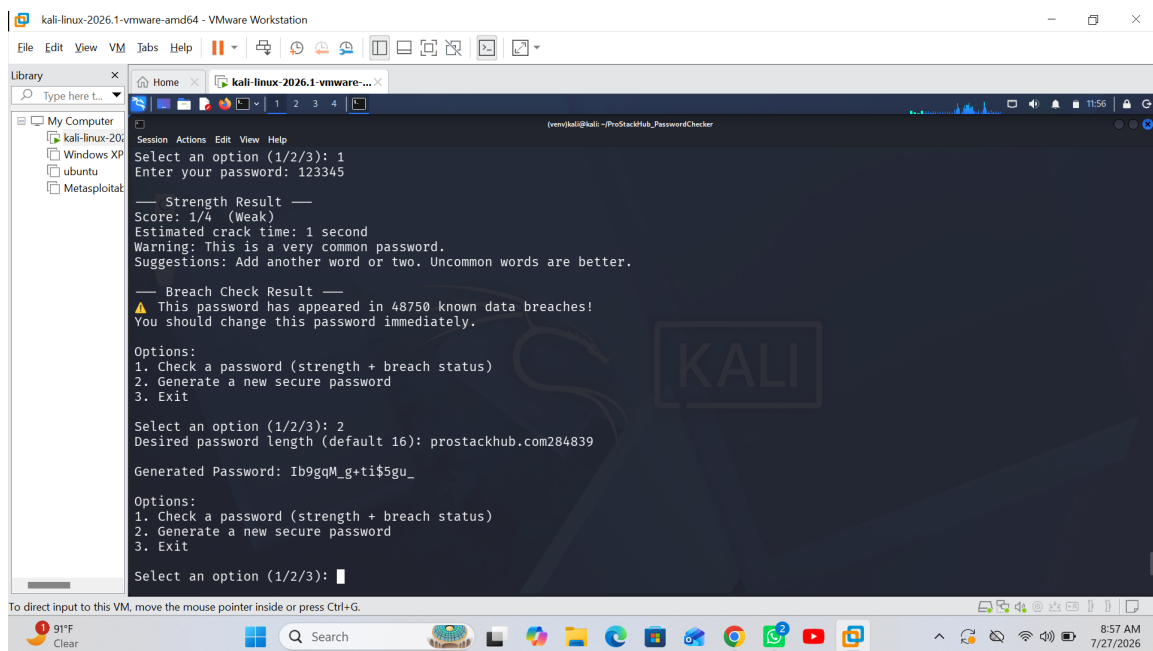
Options:
1. Check a password (strength + breach status)
2. Generate a new secure password
3. Exit
Select an option (1/2/3): 1
Enter your password: admin@19293

--- Strength Result ---
Score: 3/4 (Strong)
Estimated crack time: 14 hours

--- Breach Check Result ---
Good news: This password was not found in any known breach.

Options:
1. Check a password (strength + breach status)
2. Generate a new secure password
3. Exit
Select an option (1/2/3):
```

Screenshot: Running the application and testing a sample password. The tool correctly reports a strength score of 3/4 (Strong) with an estimated crack time, and confirms via the HaveIBeenPwned API that the password was not found in any known breach.



```
kali-linux-2026.1-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
Library
Type here to search
My Computer
kali-linux-2026.1-vmware-amd64
Windows XP
ubuntu
Metasploitat
kali-linux-2026.1-vmware-amd64
Session Actions Edit View Help
Select an option (1/2/3): 1
Enter your password: 123345

--- Strength Result ---
Score: 1/4 (Weak)
Estimated crack time: 1 second
Warning: This is a very common password.
Suggestions: Add another word or two. Uncommon words are better.

--- Breach Check Result ---
This password has appeared in 48750 known data breaches!
You should change this password immediately.

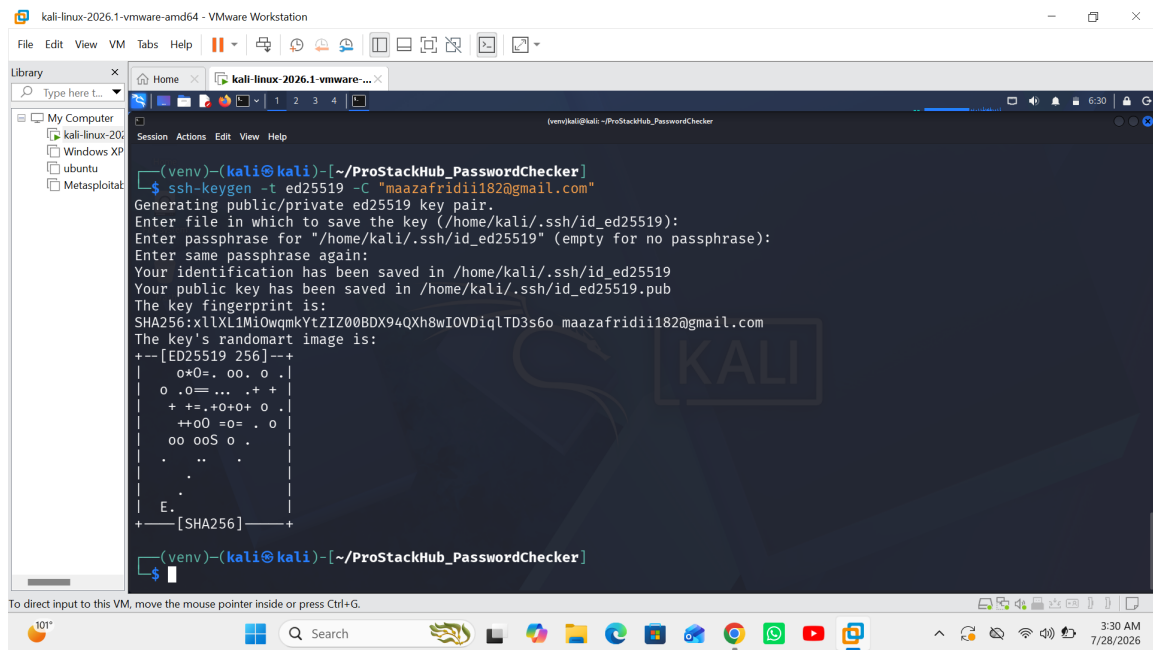
Options:
1. Check a password (strength + breach status)
2. Generate a new secure password
3. Exit
Select an option (1/2/3): 2
Desired password length (default 16): prostackhub.com284839
Generated Password: Ib9gqM_g+ti$5gu_

Options:
1. Check a password (strength + breach status)
2. Generate a new secure password
3. Exit
Select an option (1/2/3):
```

Screenshot: Testing a deliberately weak, commonly-used password. The application correctly flags it as "Very Weak" with a crack time of 1 second, and reports it was found in over 48,000 known data breaches. The secure password generator was also tested and produced a strong 16-character random password.

7. Version Control & GitHub Deployment

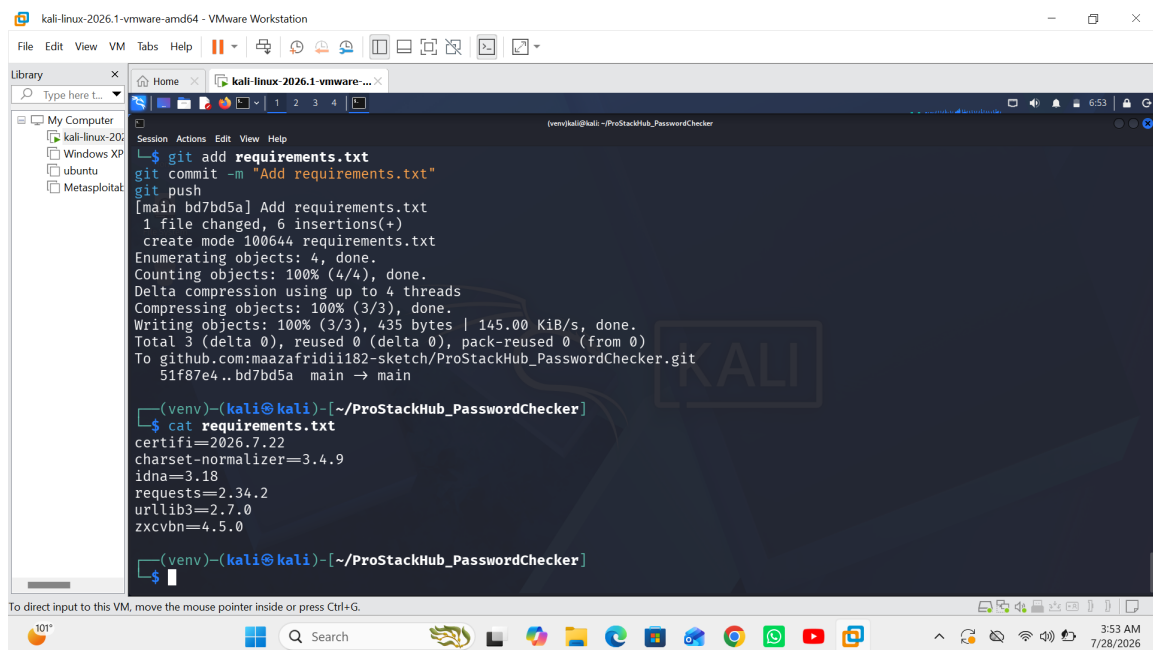
The completed project was pushed to a public GitHub repository, as required by the internship submission guidelines. SSH key-based authentication was configured on the Kali Linux machine to securely connect to GitHub without repeatedly using a password or access token.



```
(venv)-(kali@kali)-[~/ProStackHub_PasswordChecker]
$ ssh-keygen -t ed25519 -C "maazafridii182@gmail.com"
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/kali/.ssh/id_ed25519):
Enter passphrase for "/home/kali/.ssh/id_ed25519" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/kali/.ssh/id_ed25519
Your public key has been saved in /home/kali/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:xLLXLMi0WqmKtZIZ00BDX94QXh8wIOVDiqlTD3s6o maazafridii182@gmail.com
The key's randomart image is:
+--[ED25519 256]--+
  o0= . oo . o |
  o .0= ... .+ + |
  + +=.0+0+0 . |
  ++00 =0= . o |
  oo ooS o . |
  .. |
  . |
  E. |
+--[SHA256]-----+

(venv)-(kali@kali)-[~/ProStackHub_PasswordChecker]
$
```

Screenshot: Generating an SSH key pair (ED25519) on the Kali Linux machine, which was then added to the GitHub account to enable secure, password-free authentication for pushing code.



```
(venv)-(kali@kali)-[~/ProStackHub_PasswordChecker]
$ git add requirements.txt
$ git commit -m "Add requirements.txt"
$ git push
[main bd7bd5a] Add requirements.txt
1 file changed, 6 insertions(+)
create mode 100644 requirements.txt
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 4 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 435 bytes | 145.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To github.com:maazafridii182-sketch/ProStackHub_PasswordChecker.git
51f87e4..bd7bd5a main -> main

(venv)-(kali@kali)-[~/ProStackHub_PasswordChecker]
$ cat requirements.txt
certifi==2026.7.22
charset-normalizer==3.4.9
idna==3.18
requests==2.34.2
urllib3==2.7.0
zxcvbn==4.5.0

(venv)-(kali@kali)-[~/ProStackHub_PasswordChecker]
$
```

Screenshot: Final successful push of the project (including a requirements.txt file listing all dependencies) to the public GitHub repository: ProStackHub_PasswordChecker.

8. Deliverables

- Full working source code: password_checker.py
- Dependency list: requirements.txt
- Public GitHub Repository: github.com/maazafridii182-sketch/ProStackHub_PasswordChecker
- This project documentation report (PDF)

9. Conclusion

This task of the ProStackHub Cybersecurity Internship was completed successfully. The final application demonstrates a practical understanding of password security concepts, including entropy-based strength evaluation and privacy-preserving breach detection using the k-anonymity model. The project was version-controlled with Git and deployed to a public GitHub repository, following secure SSH-based authentication practices.

Submitted by: Muhammad Maaz Afridi | **Internship:** ProStackHub Cybersecurity Internship